

循环执行步骤②和③，直到 Student 表中的元组处理完为止。

(4) 哈希连接(hash join)算法

哈希连接算法把连接属性作为哈希码，用同一个哈希函数对 Student 表和 SC 表中的元组计算哈希值。具体步骤如下：

① 划分阶段，也称为创建阶段，即创建哈希表。对包含较少元组的表(如 Student 表)进行一遍处理，把它的元组按哈希函数(哈希码是连接属性)分散到哈希表的桶中。

② 试探阶段，也称为连接阶段。对另一个表(SC 表)进行一遍处理，把 SC 表的元组也按同一个哈希函数(哈希码是连接属性)进行散列，映射到相应的 Student 表哈希桶，检索该桶中的 Student 表元组，将 SC 表元组与该桶中与之相匹配的 Student 表元组连接起来。

哈希连接算法假设两个表中较小的表在第一阶段(划分阶段)后可以完全放入内存的哈希桶中。如果不满足这个前提条件，则需要对两个表进行分解，相应的哈希连接算法请参考本章文献[16]。

10.2 关系数据库管理系统的查询优化

查询优化在关系数据库管理系统中有着非常重要的地位。关系数据库系统和非过程化的 SQL 之所以能够取得巨大的成功，关键是得益于查询优化技术的发展。查询优化是影响关系数据库管理系统性能的主要因素。

优化对关系数据库管理系统来说既是挑战又是机遇。所谓挑战是指关系数据库管理系统为了达到用户可接受的性能必须进行查询优化。由于关系表达式的语义级别很高，使关系数据库管理系统可以从关系表达式中分析查询语义，从而提供了执行查询优化的可能性。这就为关系数据库管理系统在性能上接近甚至超过非关系数据库管理系统提供了机遇。

10.2.1 查询优化概述

查询优化既是关系数据库管理系统实现的关键技术，又是其优点所在。它减轻了用户选择存取路径的负担。用户只要提出“干什么”，而不必指出“怎么干”。对比一下非关系数据库管理系统中的情况：用户使用过程化的语言表达查询要求，至于执行何种记录级的操作以及操作的序列，是由用户而不是由系统来决定的。因此用户必须了解存取路径，系统要提供给用户选择存取路径的手段，查询效率由用户的存取策略决定。如果用户做了不当的选择，系统是无法对此加以改进的。这就要求用户有较高的数据库技术和程序设计水平。

查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较高的效率，而且在于系统可以比用户程序的“优化”做得更好。这是因为：

① 优化器可以从数据字典中获取许多统计信息，例如每个关系表中的元组数、关系中每个属性值的分布情况、哪些属性上已经建立了索引等。优化器可以根据这些信息做出正确的估算，选择高效的执行计划，而用户程序则难以获得这些信息。

② 如果数据库的物理统计信息改变了,系统可以自动对查询进行重新优化以选择相适应的执行计划。在非关系数据库管理系统中则必须重写程序,而重写程序在实际应用中往往是不太可能的。

③ 优化器可以考虑数百种甚至数千种不同的执行计划,而程序员一般只能考虑有限的几种可能性。

④ 优化器中包含很多复杂的优化技术,这些优化技术往往只有最好的程序员才能掌握。系统的自动优化相当于使得所有人都拥有这些优化技术。

关系数据库管理系统通过某种代价模型计算出各种查询执行策略的执行代价,然后选取代价最小的执行方案。在集中式数据库中,查询执行开销主要包括磁盘存取块数(I/O 代价)、处理机时间(CPU 代价)以及查询的内存代价。在分布式数据库中还要加上通信代价,即

$$\text{总代价} = \text{I/O 代价} + \text{CPU 代价} + \text{内存代价} + \text{通信代价}$$

由于磁盘 I/O 操作涉及机械动作,需要的时间与内存操作相比要高几个数量级,因此在计算查询代价时一般用查询处理读写的块数作为衡量单位。

查询优化的总目标是:选择有效的策略,求得给定关系表达式的值,使得查询代价较小。因为查询优化的搜索空间有时非常大,实际系统选择的策略不一定是最优的,而是较优的。

10.2.2 一个实例

首先通过一个简单的例子来说明为什么要进行查询优化。

[例 10.3] 求选修了 81003 课程的学生姓名。

用 SQL 语句表达如下:

```
SELECT Student.Sname
FROM Student, SC
WHERE Student.Sno = SC.Sno AND SC.Cno = '81003';
```

假定“学生选课管理”数据库中有 1 000 个学生记录,10 000 个选课记录,其中选修 81003 课程的选课记录为 50 个。

系统可以用多种等价的关系代数表达式来完成这一查询,但分析下面三种情况就足以说明问题:

$$Q_1 = \Pi_{\text{Sname}}(\sigma_{\text{Student.Sno}=\text{SC.Sno} \wedge \text{SC.Cno}='81003'}(\text{Student} \times \text{SC}))$$

$$Q_2 = \Pi_{\text{Sname}}(\sigma_{\text{SC.Cno}='81003'}(\text{Student} \bowtie \text{SC}))$$

$$Q_3 = \Pi_{\text{Sname}}(\text{Student} \bowtie \sigma_{\text{SC.Cno}='81003'}(\text{SC}))$$

后面将看到由于查询执行的策略不同,查询效率相差很大。

1. 第一种情况

① 计算广义笛卡儿积。把 Student 表和 SC 表的每个元组连接起来。其常用算法是:先在

内存中尽可能多地装入某个表(如 Student 表)的若干块,留出一块存放另一个表(如 SC 表)的元组,然后把读入的 SC 表中的每个元组与内存中的 Student 表的每个元组连接,连接后的元组装满一块后就写到中间文件上,再从 SC 表中读入一块与内存中的 Student 表的元组连接,直到 SC 表处理完;这时再一次读入若干块 Student 元组,读入一块 SC 元组,重复上述处理过程,直到把 Student 表处理完。

设一个数据块能装 10 个 Student 元组或 100 个 SC 元组,在内存中存放 5 块 Student 元组和 1 块 SC 元组,则读取总块数为

$$\frac{1\,000}{10} + \frac{1\,000}{10 \times 5} \times \frac{10\,000}{100} = 100 + 20 \times 100 = 2\,100 \text{ 块}$$

其中,读 Student 表 100 块,读 SC 表 20 遍,每遍 100 块,则总计要读取 2 100 个数据块。连接后的元组数为 $10^3 \times 10^4 = 10^7$ 。设每块能装 10 个元组,则写出 10^6 块。

② 做选择操作。依次读入连接后的元组,按照选择条件选取满足要求的记录。假定内存处理时间忽略。这一步读取中间文件的数量(同写中间文件一样)是 10^6 块。若满足条件的元组假设仅 50 个,则均可放在内存。

③ 做投影操作。把第②步的结果在 Sname 上做投影输出,得到最终结果。

因此第一种情况下执行查询的总读写数据块 = $2\,100 + 10^6 + 10^6$ 。

2. 第二种情况

① 计算自然连接。为了执行自然连接,读取 Student 表和 SC 表的策略不变,总的读取块数仍为 2 100 块。但自然连接的结果比第一种情况大大减少,连接后的元组数为 10^4 个元组,写出数据块 = 10^3 块。

② 读取中间文件块,执行选择操作,读取的数据块 = 10^3 块。

③ 把第②步结果投影输出。

第二种情况下执行查询的总读写数据块 = $2\,100 + 10^3 + 10^3$ 。其执行代价大约是第一种情况的 $1/488$ 。

3. 第三种情况

① 先对 SC 表做选择操作,只需读一遍 SC 表,存取块数为 100 块,因为满足条件的元组仅 50 个,不必使用中间文件。

② 读取 Student 表,把读入的 Student 元组和内存中的 SC 元组做连接。也只需读一遍 Student 表,共 100 块。

③ 把连接结果投影输出。

第三种情况下执行查询总的读写数据块 = $100 + 100$ 。其执行代价大约是第一种情况的 $1/10\,000$,是第二种情况的 $1/20$ 。

假如 SC 表的 Cno 字段上有索引,第一步就不必读取所有的 SC 元组,而只需读取 Cno = '81003' 的那些元组(50 个)。存取的索引块和 SC 中满足条件的数据块共 3~4 块。若 Student 表在 Sno 上也有索引,则第二步也不必读取所有的 Student 元组,因为满足条件的 SC 记录仅 50

条, 涉及最多 50 条 Student 记录, 因此读取 Student 表的块数也可大大减少。

这个简单的例子充分说明了查询优化的必要性, 同时也给出一些查询优化方法的初步概念。例如, 读者可能已经发现, 在第一种情况下连接后的元组可以先不立即写出, 而是和第②步的选择操作结合, 这样可以省去写出和读入的开销。有选择和连接操作时, 可以先做选择操作, 例如把上面的代数表达式 Q_1 、 Q_2 变换为 Q_3 , 这样参加连接的元组就可以大大减少, 这是代数优化。在 Q_3 中, SC 表的选择操作算法可以采用全表扫描或索引扫描, 经过初步估算, 索引扫描方法较优。同样, 对于 Student 和 SC 表的连接, 利用 Student 表上的索引, 采用索引连接代价也较小, 这就是物理优化。

10.3 代数优化

SQL 语句经过查询分析和查询检查后变换为语法树, 它是关系代数表达式的内部表示。本节介绍基于关系代数等价变换规则的优化方法, 即代数优化, 也称作逻辑优化。

10.3.1 关系代数表达式等价变换规则

代数优化策略是通过对关系代数表达式的等价变换来提高查询效率。所谓关系代数表达式的等价, 是指用相同的关系统代替两个表达式中相应的关系所得到的结果是相同的。两个关系表达式 E_1 和 E_2 是等价的, 可记为 $E_1 \equiv E_2$ 。

下面是常用的等价变换规则, 证明从略。

1. 连接运算、笛卡儿积运算的交换律

设 E_1 和 E_2 是关系代数表达式, F 是连接运算的条件, 则有

$$\begin{aligned} E_1 \times E_2 &\equiv E_2 \times E_1 \\ E_1 \bowtie E_2 &\equiv E_2 \bowtie E_1 \\ E_1 \bowtie_F E_2 &\equiv E_2 \bowtie_F E_1 \end{aligned}$$

2. 连接运算、笛卡儿积运算的结合律

设 E_1 、 E_2 、 E_3 是关系代数表达式, F_1 和 F_2 是连接运算的条件, 则有

$$\begin{aligned} (E_1 \times E_2) \times E_3 &\equiv E_1 \times (E_2 \times E_3) \\ (E_1 \bowtie E_2) \bowtie E_3 &\equiv E_1 \bowtie (E_2 \bowtie E_3) \\ (E_1 \bowtie_{F_1} E_2) \bowtie_{F_2} E_3 &\equiv E_1 \bowtie_{F_1} (E_2 \bowtie_{F_2} E_3) \end{aligned}$$

3. 投影运算的串接律

$$\Pi_{A_1, A_2, \dots, A_n} (\Pi_{B_1, B_2, \dots, B_m} (E)) \equiv \Pi_{A_1, A_2, \dots, A_n} (E)$$

其中, E 是关系代数表达式, $A_i (i=1, 2, \dots, n)$, $B_j (j=1, 2, \dots, m)$ 是属性名, 且 $\{A_1, A_2, \dots, A_n\}$ 是 $\{B_1, B_2, \dots, B_m\}$ 的子集。

4. 选择运算的串接律

$$\sigma_{F_1} (\sigma_{F_2} (E)) \equiv \sigma_{F_1 \wedge F_2} (E)$$